

2 · MARC Architecture

*The core contribution. Same PPO training as vanilla, but the network now (a) summarises its **own recent role** from a history window, (b) **infers a role for each teammate**, and (c) **attention-pools** those teammate roles into the policy. $LAMBDA_AUX = 0$ here — no auxiliary loss at all, so every gain on this page is purely architectural.*

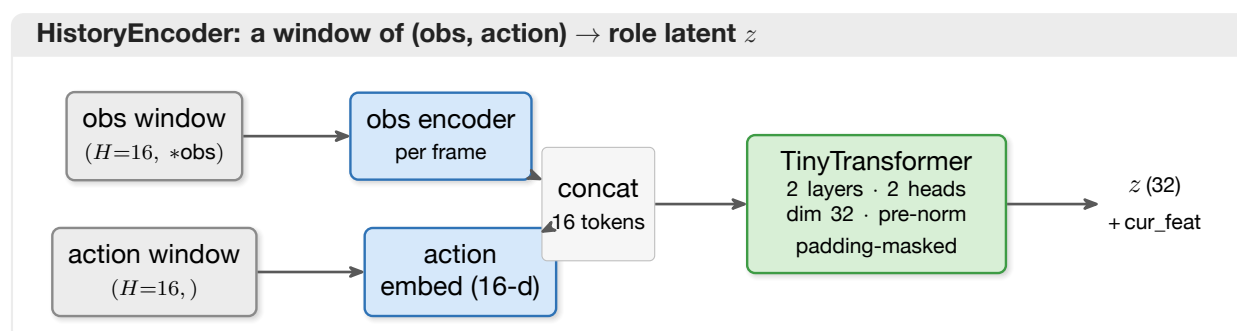
The colour key

Every box in every diagram is coloured by *what kind of component it is*. Same kind, same colour, everywhere.

INPUT env tensors **ENCODER** obs $\rightarrow$ features **TRANSFORMER** history self-attention **X-ATTN** pool over teammates **HEAD** policy / value **LOSS** aux term / gate

The one reusable block: HistoryEncoder

Everything is built from a single block. It turns the *last* $H=16$ steps an agent saw and did into one 32-number **role latent** z — a compressed answer to “*what role has this agent been playing lately?*”



The whole project is just *how many copies of this block you run and what you do with the z 's*.

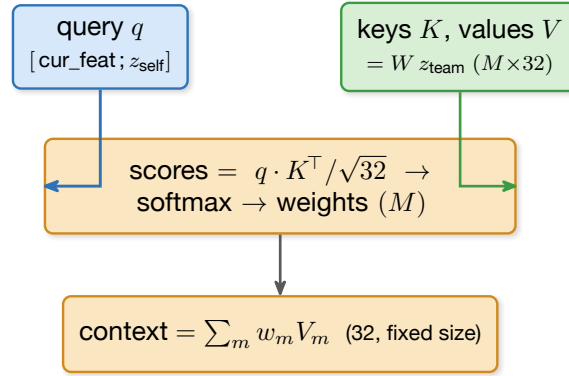
Three new ideas

1. **Self-encoder** — “what role have *I* been playing?” Run HistoryEncoder on my own window $\rightarrow z_{\text{self}}$.
2. **Per-teammate inferencer** — “what role is *each* teammate playing?” A *separate* HistoryEncoder (own weights), applied once per teammate slot, weight-shared across slots $\rightarrow z_{\text{team}}$ of shape $(M, 32)$.
3. **Attention pool** — the policy can’t eat a variable number of teammate vectors, so we use **cross-attention** to squeeze them into one fixed-size context. With $M=1$ (Overcooked) it degrades to the identity; with $M=8$ (MPE $N=9$) it works unchanged.

What cross-attention does here

A *query* (me) asks “which teammates matter to my decision right now?” and gets a relevance-weighted blend of their roles.

Cross-attention pool: 1 query (me) over M teammate roles

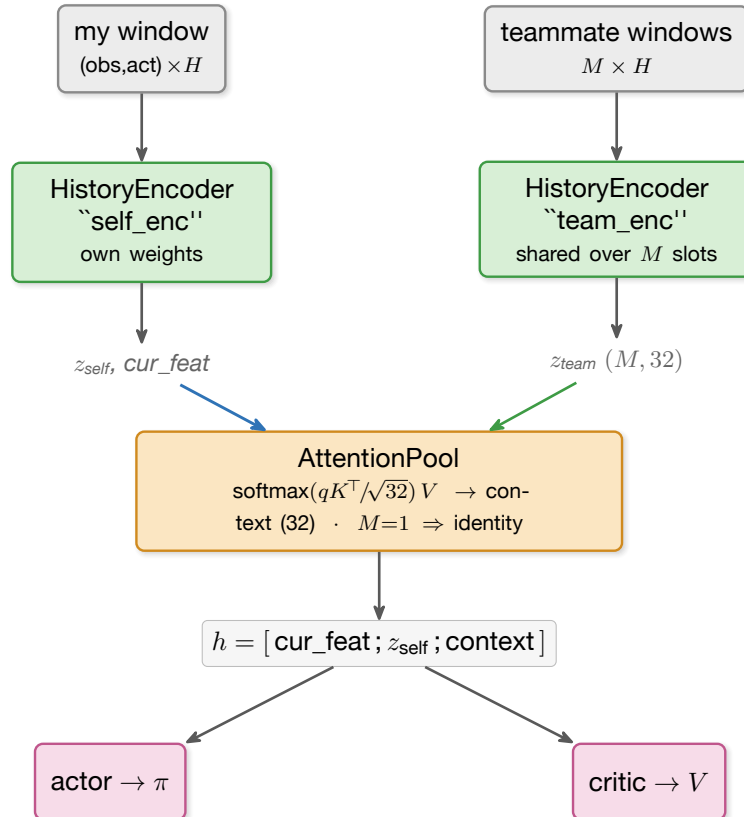


Worked example ($N=3$, so $M=2$ teammates)

Teammate A is “covering the left landmark”, B “covering centre”, I’m by the right one. Dot-products give scores $[A: 0.3, B: 2.1] \rightarrow \text{softmax} [0.14, 0.86] \rightarrow \text{context} \approx \text{mostly B's role: “B's plan most constrains mine, so weight B.”}$ With $M=1$, softmax of one element = $[1.0] \rightarrow \text{context} = \text{that teammate's value, unchanged}$ — which is why the same code runs Overcooked ($M=1$) and MPE $N=9$ ($M=8$).

The full network

MARC architecture — teammate-aware policy (no aux loss)



Speed note. During rollout the two aux-only encoder passes are skipped (`compute_aux=False`); they're only needed by the loss in file 3, and skipping them roughly halves encoder cost when collecting data.

Why this beats vanilla

The three interns from file 1 now each keep a short diary of what they’ve been doing (z_{self}) and a guess of each colleague’s role (z_{team}), and consult “whose plan most affects mine” before acting. Even with **nobody telling them to specialise**, just *seeing* teammate roles stops them colliding on the obvious task.

The gap that motivates file 3

We *measure* the teammate role latents and they are still highly **similar** ($\text{role_sim} \approx +0.95$ at $N=6/9$). The architecture lets agents *see* roles but does not *push* them apart — they de-conflict largely by luck of initial position. Closing that is the auxiliary loss’s job.

Results

MPE `simple_spread`, eval return (higher is better; reward is negative). `role_sim` = mean teammate-pair latent cosine ($\approx +1 \Rightarrow$ redundant roles, $\approx 0 \Rightarrow$ distinct). Seeds 30–32 are the original confirmation; 33+ are the power-up sweep (job 1565450) — table grows as they land.

N	n	mean	sd	role_sim	per-seed
3	8	-10.95	0.34	+0.55	s30 -11.27, s31 -10.49, s32 -11.06, s33 -11.41, s34 -11.08, s35 -10.47, s36 -10.66, s37 -11.17
6	8	-31.67	0.58	+0.96	s30 -31.06, s31 -31.09, s32 -32.74, s33 -31.72, s34 -31.04, s35 -31.79, s36 -32.29, s37 -31.61
9	8	-43.42	1.69	+0.95	s30 -41.19, s31 -42.26, s32 -46.21, s33 -44.25, s34 -44.78, s35 -41.84, s36 -44.79, s37 -42.06

vs vanilla: +8.60 ($N=3$) · +3.06 ($N=6$) · +10.81 ($N=9$). A robust win at every team size. The $N=9$ sd (2.16) comes from one bad seed (s32, -46.2) — the auxiliary loss in file 3 specifically removes that instability.